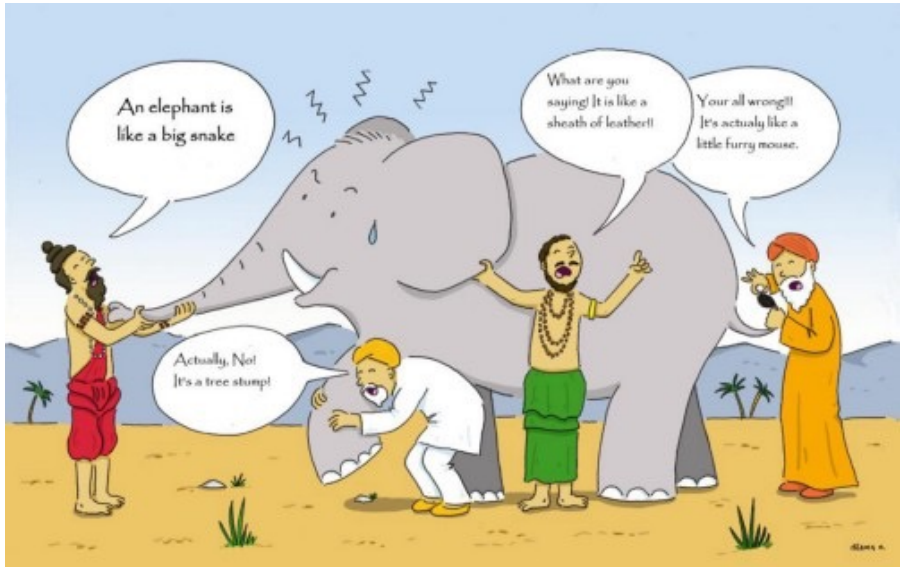
南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY
地球与空间科学系
DEPARTMENT OF EARTH AND SPACE SCIENCES

Lesson 8: Ensemble learning

CHEN Kejie 陈克杰（chenkj@sustech.edu.cn）

30-Mar-26



Ensemble learning

我们已经学习了很多“单一模型”：
•线性模型：感知机、**Logistic** 回归、**SVM**
•非线性模型：决策树
•概率模型：贝叶斯分类器
这些方法都是：设计一个分类器，然后训练它，独自作出判断。

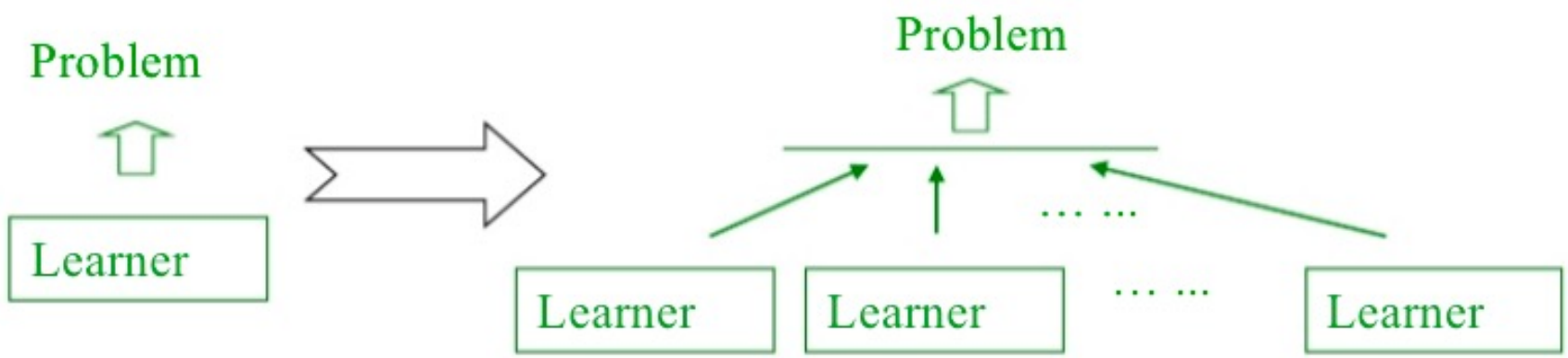
但有没有一种可能：多个弱分类器结合起来，能做得更好？
可以引出这样一个思考题或真实案例：
•假设有三个医生，分别判断一个病人是否得病：

- 医生**A**准确率**70%**，医生**B**准确率**72%**，医生**C**准确率**68%**
- 单独每个都不是很强，但如果“多数表决”，准确率可能更高？

这个思想就是—集体的智慧可能优于单一决策者。
机器学习中也一样，很多模型“联合投票”，可以提升性能、稳定性和泛化能力。

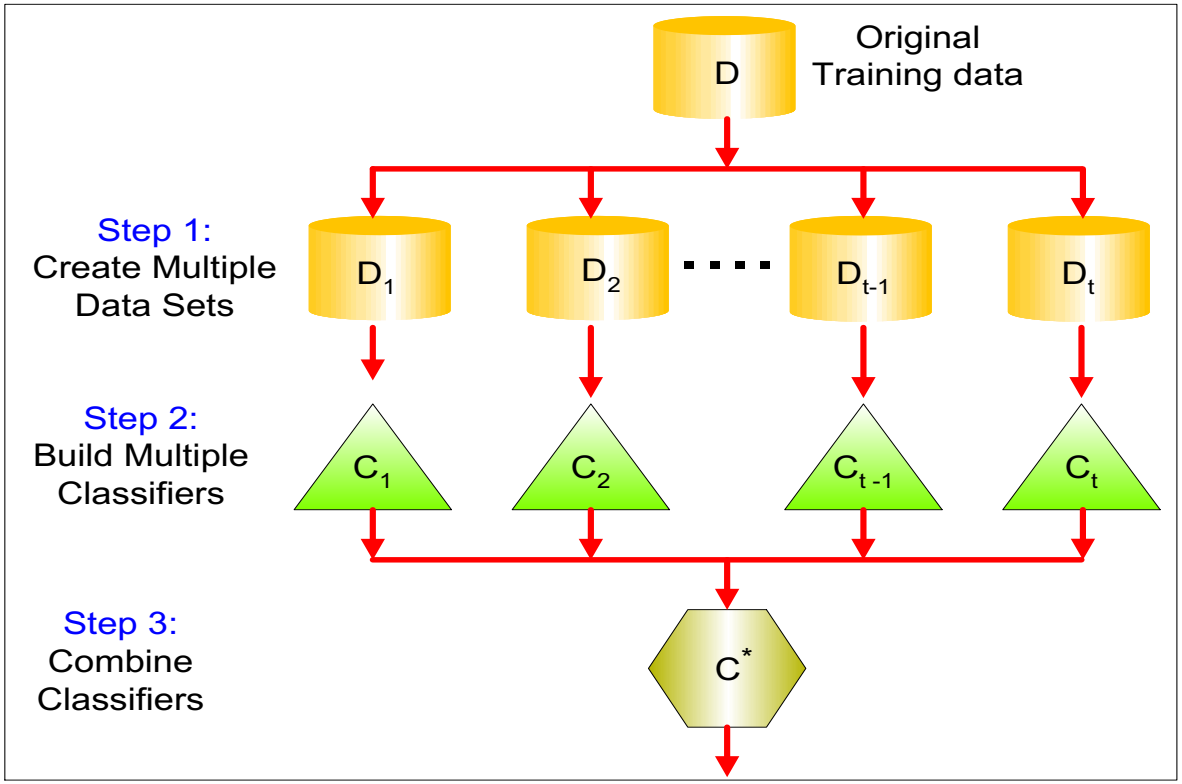
Ensemble learning

Ensemble learning completes learning tasks by building and combining multiple learners (通过构建并结合多个学习器来完成学习任务)。



- **Homogeneous integration （同质集成）:** the integration contains only the same type of "individual learners" and the corresponding learning algorithm is called base learning algorithm
  Individual learner also known as base learner
- **Heterogeneous integration （异质集成）:** individual learners are generated by different learning algorithms and there is no base learning algorithm

Why Ensemble?



- **Suppose there are 25 base classifiers**
- Each classifier has error rate, $\varepsilon = 0.35$
- Assume classifiers are independent
- Probability that the ensemble classifier makes a wrong prediction:

$$\sum_{i=13}^{25} \binom{25}{i} \varepsilon^{i} (1-\varepsilon)^{25-i} = 0.06$$

How to get a good ensemble?

	测试例1	测试例2	测试例3
h_1	√	√	×
h_2	×	√	√
h_3	√	×	√
集成	√	√	√

(a) 集成提升性能

	测试例1	测试例2	测试例3
h_1	√	√	×
h_2	√	√	×
h_3	√	√	×
集成	√	√	×

(b) 集成不起作用

	测试例1	测试例2	测试例3
h_1	√	×	×
h_2	×	√	×
h_3	×	×	√
集成	×	×	×

(c) 集成起负作用

Make individual learners good but different (好而不同)

In reality, ensemble learning technology is widely used in various machine learning and data mining applications

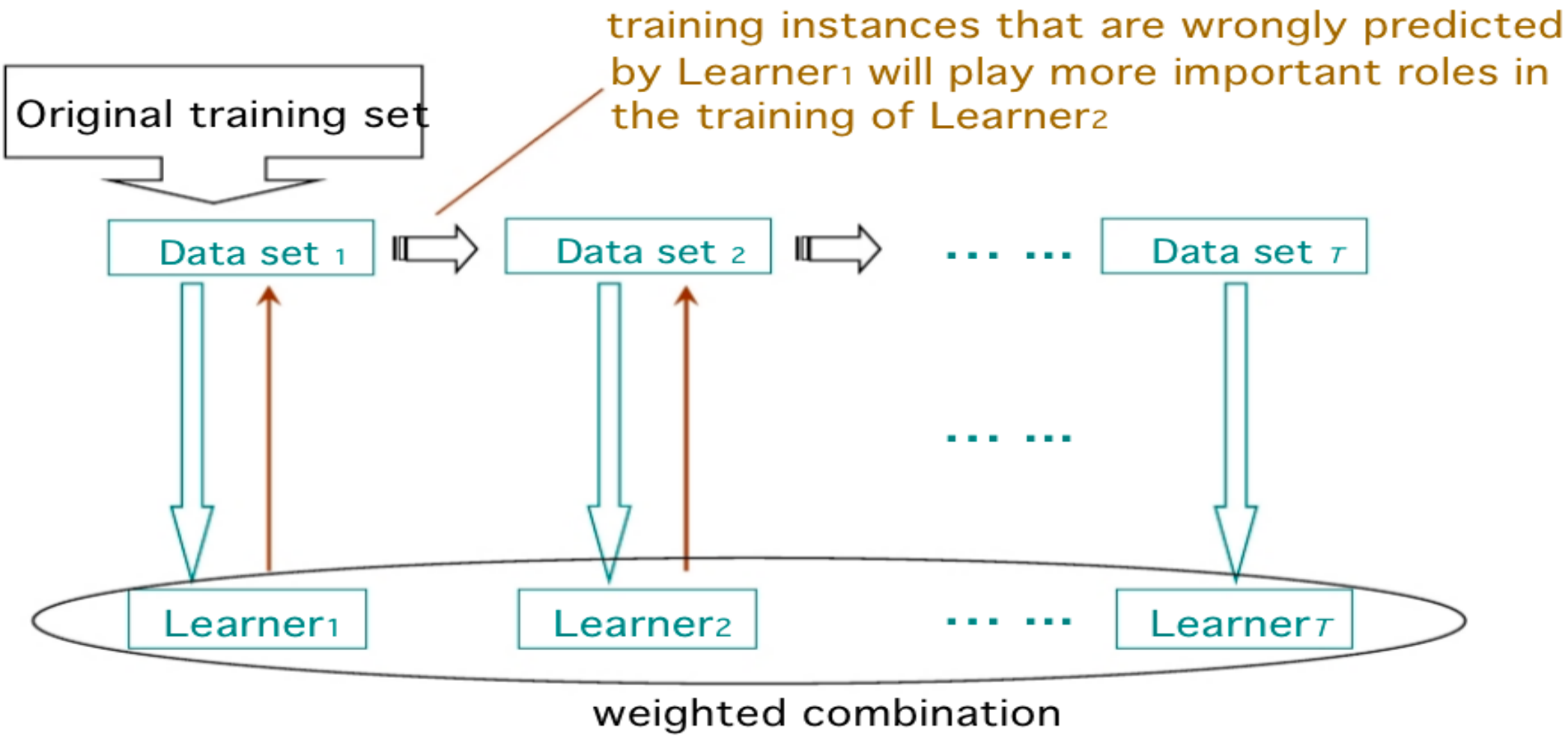
想获胜，用集成

Many successful ensemble learning methods

- **Serialization method （序列化方法）**
  - AdaBoost [Freund & Schapire, JCSS97]
  - GradientBoost [Friedman, AnnStat01]
  - LPBoost [Demiriz, Bennett, Shawe-Taylor, MLJ06]
  - … …

- **Parallelization method （并行化方法）**
  - Bagging [Breiman, MLJ96]
  - Random Forest [Breiman, MLJ01]
  - Random Subspace [Ho, TPAMI98]
  - … …

Boosting: A flowchart illustration



Boosting

- Equal weights are assigned to each training instance (1/N for round 1) at first round
- After a classifier C_i is learned, the weights are adjusted to allow the subsequent classifier

C_{i+1} to "pay more attention" to data that were misclassified by C_i.

- Final boosted classifier C^* combines the votes of each individual classifier
  - Weight of each classifier's vote is a function of its accuracy
- Adaboost – popular boosting algorithm

Adaboost (Adaptive Boost 自适应增强)

- Input:
  - Training set *D* containing ***N*** instances
  - *T* rounds
  - A classification learning scheme
- Output:
  - A composite model

Adaboost training phase

- Training data D contain N labeled data (X_1,y_1), (X_2,y_2), (X_3,y_3),….(X_N,y_N)
- Initially assign equal weight 1/d to each data
- To generate *T* base classifiers, we need *T* rounds or iterations
- Round i, data from *D* are sampled with replacement , to form Di (size *N*)
- Each data's chance of being selected in the next rounds depends on its weight
  - Each time the new sample is generated directly from the training data *D* with different sampling probability according to the weights; these weights are not zero

对于允许带权样本的基学习算法，这一步不是必须的

Adaboost training phase

- Base classifier C_i, is derived from training data of D_i
- Error of C_i is tested using D_i
- Weights of training data are adjusted depending on how they were classified
- Correctly classified: Decrease weight
- Incorrectly classified: Increase weight
- Weight of a data indicates how hard it is to classify it (directly proportional)

Adaboost testing phase

- The lower a classifier error rate, the more accurate it is, and therefore, the higher its weight for voting should be
- Weight of a classifier C_i' s vote is

$$\alpha_i = \frac{1}{2}\ln\left(\frac{1-\varepsilon_i}{\varepsilon_i}\right)$$

- Testing:
- For each class c, sum the weights of each classifier that assigned class c to X (unseen data)
- The class with the highest sum is the WINNER!

$$C^*(x_{test}) = \arg\max_y \sum_{i=1}^{T} \alpha_i \delta\left(C_i(x_{test}) = y\right)$$

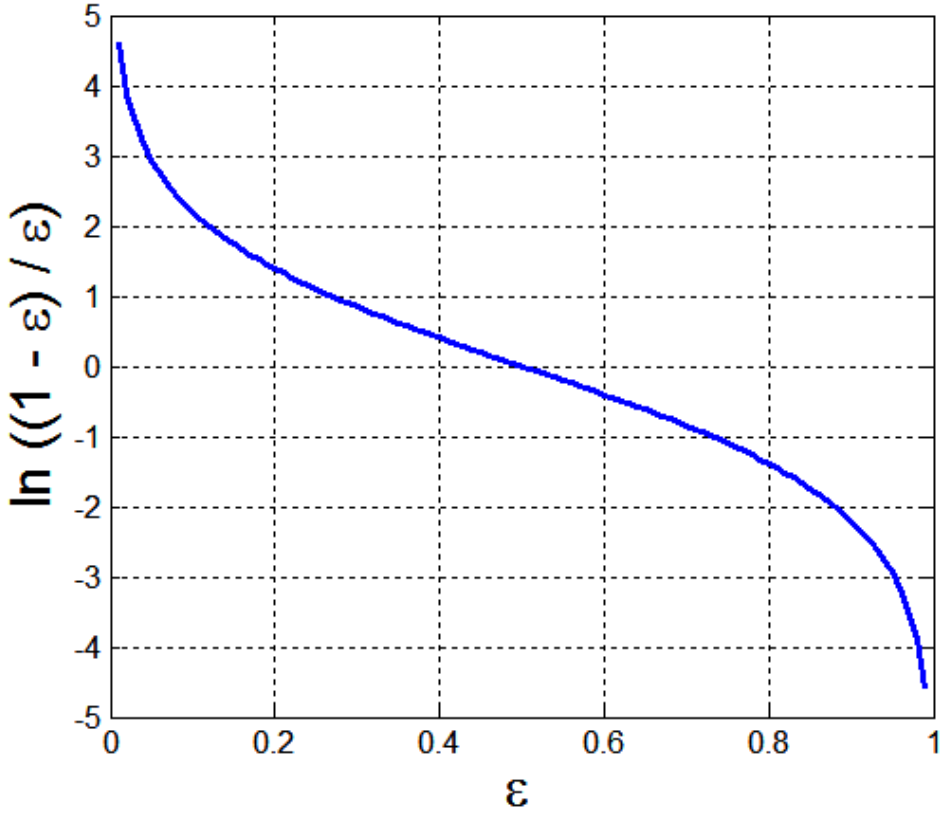
Example: Error and Classifier Weight in AdaBoost

- **Base classifiers: $C_1, C_2, \ldots, C_T$**
- **Error rate: (*i* = index of classifier, *j*=index of instance)**

$$\varepsilon_i = \frac{1}{N}\sum_{j=1}^{N} w_j \delta\left(C_i(x_j) \neq y_j\right)$$

- **Importance of a classifier:**

$$\alpha_i = \frac{1}{2}\ln\left(\frac{1-\varepsilon_i}{\varepsilon_i}\right)$$



当且仅当基分类器的错误率 **εt>0.5**，也就是比随机猜还差时，不符合弱分类器的定义，可以丢弃

Adaboost (Adaptive Boost 自适应增强)

给定如下表所示训练数据，假设弱分类器由x<v或x>v （单层决策树，决策树桩）产生，其阈值v 使该分类器在训练数据集上分类误差率最低。试用 AdaBoost 算法学习一个强分类器。

序号	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	–1	–1	–1	1	1	1	–1

解 初始化数据权值分布

$$D_1 = (w_{11}, w_{12}, \cdots, w_{110})$$
$$w_{1i} = 0.1, \quad i = 1, 2, \cdots, 10$$

对 $m = 1$，

（a）在权值分布为 D_1 的训练数据上，阈值 v 取 2.5 时分类误差率最低，故基本分类器为

$$G_1(x) = \begin{cases} 1, & x < 2.5 \\ -1, & x > 2.5 \end{cases}$$

Adaboost (Adaptive Boost 自适应增强)

(b) 计算弱分类器误差率

$$e_1 = P(G_1(x_i) \neq y_i) = 0.3$$

训练集共有 10 个样本，其中 G_1 分错了 3 个点，且初始权重都为 0.1，
所以加权误差率为：

$$e_1 = 3 \times 0.1 = 0.3$$

(c) 计算分类器系数

$$\alpha_1 = \frac{1}{2} \log \frac{1 - e_1}{e_1} = \frac{1}{2} \log \frac{0.7}{0.3} = 0.4236$$

Adaboost (Adaptive Boost 自适应增强)

(d) 更新训练样本权值分布

权值更新公式：

$$w_{2i} = \frac{w_{1i}}{Z_1} \exp(-\alpha_1 y_i G_1(x_i)), \quad i = 1, 2, \ldots, 10$$

其中归一化因子：

$$Z_1 = \sum_{i=1}^{10} w_{1i} \exp(-\alpha_1 y_i G_1(x_i))$$

由于：

- 若样本被 **正确分类**，则 $y_i G_1(x_i) = 1$，故

$$w_{2i} \propto 0.1e^{-\alpha_1}$$

- 若样本被 **错误分类**，则 $y_i G_1(x_i) = -1$，故

$$w_{2i} \propto 0.1e^{\alpha_1}$$

又因为本轮：

- 正确分类 7 个样本
- 错误分类 3 个样本

所以

$$Z_1 = 7 \cdot 0.1e^{-\alpha_1} + 3 \cdot 0.1e^{\alpha_1} \approx 0.9165$$

进一步得到：

对于正确分类样本

$$w_{2i} = \frac{0.1e^{-0.4236}}{0.9165} \approx 0.07143$$

对于错误分类样本

$$w_{2i} = \frac{0.1e^{0.4236}}{0.9165} \approx 0.16667$$

因此新的权值分布为：

$$D_2 = (0.07143, 0.07143, 0.07143, 0.07143, 0.07143, 0.07143, 0.16667, 0.16667, 0.16667, 0.07143)$$

即：**分错样本的权重升高，分对样本的权重降低。**

Adaboost (Adaptive Boost 自适应增强)

对 $m=2$，

（a）在权值分布为 D_2 的训练数据上，阈值 v 是 8.5 时分类误差率最低，基本分类器为

$$G_2(x)=\begin{cases}1, & x<8.5\\ -1, & x>8.5\end{cases}$$

（b） $G_2(x)$ 在训练数据集上的误差率 $e_2=0.2143$.

（c）计算 $\alpha_2=0.6496$.

（d）更新训练数据权值分布：

$$D_3=(0.0455, 0.0455, 0.0455, 0.1667, 0.1667, 0.1667, 0.1060, 0.1060, 0.1060, 0.0455)$$

$$f_2(x)=0.4236G_1(x)+0.6496G_2(x)$$

分类器 $\mathrm{sign}[f_2(x)]$ 在训练数据集上有 3 个误分类点.

Adaboost (Adaptive Boost 自适应增强)

对 $m=3$，

（a）在权值分布为 D_3 的训练数据上，阈值 v 是 5.5 时分类误差率最低，基本分类器为

$$G_3(x)=\begin{cases}1, & x>5.5\\ -1, & x<5.5\end{cases}$$

此时是有4个样本被分错

（b）$G_3(x)$ 在训练样本集上的误差率 $e_3=0.1820$.

（c）计算 $\alpha_3=0.7514$.

（d）更新训练数据的权值分布：

$$D_4=(0.125,0.125,0.125,0.102,0.102,0.102,0.065,0.065,0.065,0.125)$$

于是得到：

Adaboost (Adaptive Boost 自适应增强)

$$f_3(x) = 0.4236G_1(x) + 0.6496G_2(x) + 0.7514G_3(x)$$

分类器 $\text{sign}[f_3(x)]$ 在训练数据集上误分类点个数为 0.

于是最终分类器为

$$G(x) = \text{sign}[f_3(x)] = \text{sign}[0.4236G_1(x) + 0.6496G_2(x) + 0.7514G_3(x)]$$

sign[f(x)]是停止的条件

Adaboost (Adaptive Boost 自适应增强)

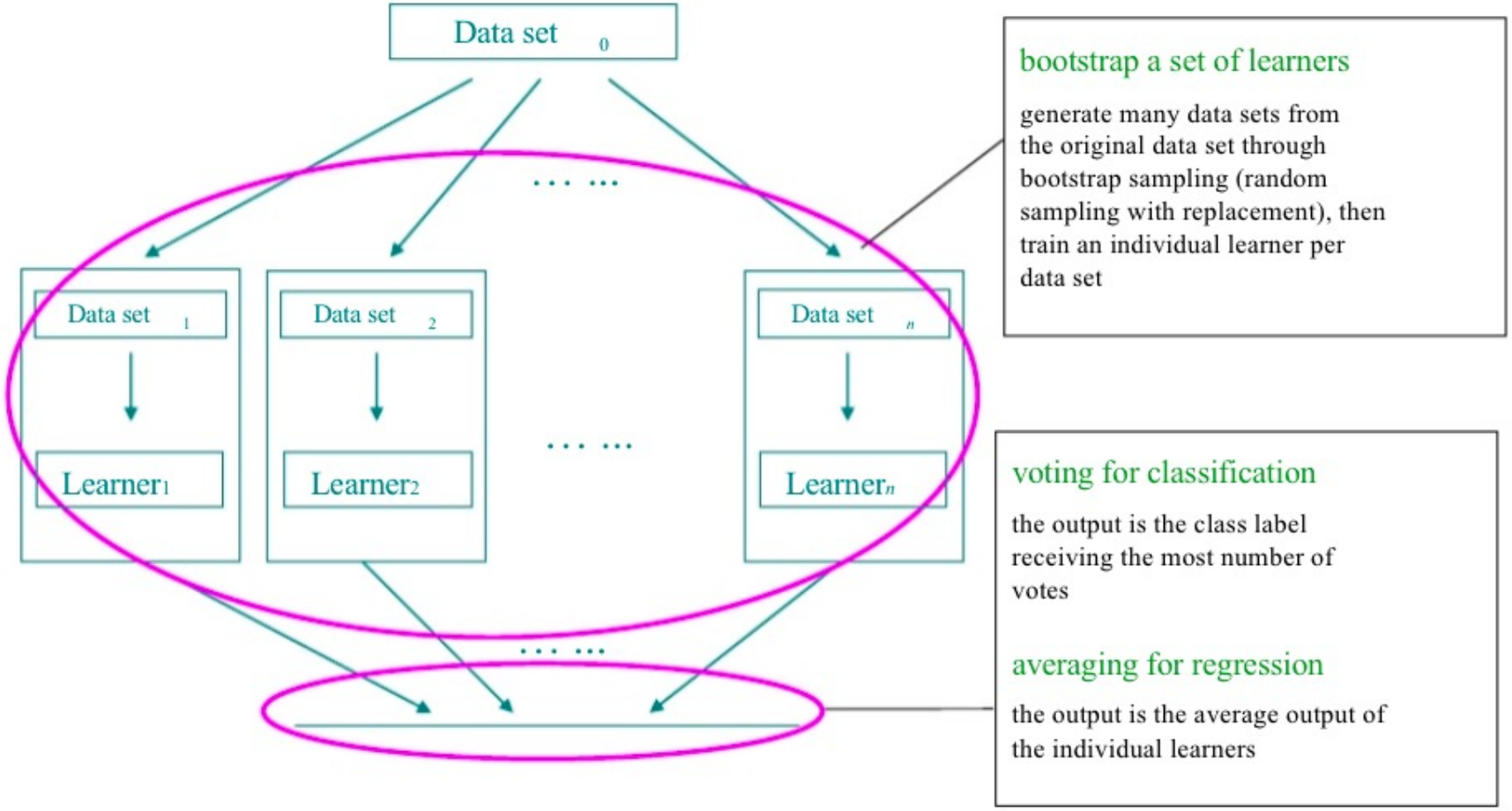
Adaboost算法人脸检测

人脸检测的目标是找出图像中的所有人脸存在的位置

主流的人脸检测方法基于以上直方图特征、颜色特征、模板特征、结构特征及Haar特征等特征采用**Adaboost学习算法**，Adaboost算法是一种用来分类的方法，它把一些比较弱的分类方法合在一起，组合出新的很强的分类方法。

Bagging

Adaboost，容易受噪声影响，尤其在样本质量不高时；与之互补的方法—**Bagging**，它不再‘惩罚’错误样本，而是通过引入‘多样性’的方式让模型更稳定。



Bagging

- Sampling with replacement

Training Data

Data ID

Original Data	1	2	3	4	5	6	7	8	9	10
Bagging (Round 1)	7	8	10	8	2	5	10	10	5	9
Bagging (Round 2)	1	4	9	1	2	3	2	7	3	2
Bagging (Round 3)	1	8	5	10	5	5	9	6	3	7

- Build classifier on each bootstrap sample
- Each sample has probability $(1 – 1/n)^n$ of being selected as test data
- Training data = $1- (1 – 1/n)^n$ of the original data

The 0.632 bootstrap

- This method is also called the *0.632 bootstrap*
  - A particular training data has a probability of 1-1/*n* of *not* being picked
  - Thus its probability of ending up in the test data (not selected) is:
  - This means the training data will contain approximately 63.2% of the instances

Random Forests

- **Ensemble method specifically designed for decision tree classifiers**
- **Random Forests grows many trees**
- Ensemble of unpruned decision trees
- Each base classifier classifies a “new” vector of attributes from the original data
- Final result on classifying a new instance: voting. Forest chooses the classification result having the most votes (over all the trees in the forest)

随机森林 = **Bagging +** 决策树 **+** 随机特征选择
抑制过拟合能力强，特别适合高维数据

Random Forests

- **Introduce two sources of randomness: “Bagging” and “Random input vectors”**
- Bagging method: each tree is grown using a bootstrap sample of training data
- Random vector method: At each node, best split is chosen from a random sample of *m* attributes instead of all attributes

$m = log_2 d$

构造随机森林的 4 个步骤

1 抽取 N 个样本：一个样本容量为 N 的样本，有放回的抽取 N 次，每次抽取 1 个，最终形成了 N 个样本。这选择好了的 N 个样本用来训练一个决策树，作为决策树根节点处的样本。

2 选择 m 个属性：当每个样本有 M 个属性时，在决策树的每个节点需要分裂时，随机从这 M 个属性中选取出 m 个属性，满足条件 m << M 。然后从这 m 个属性中采用某种策略（比如说信息增益）来选择 1 个属性作为该节点的分裂属性。

3 构造决策树：决策树形成过程中每个节点都要按照步骤 2 来分裂（很容易理解，如果下一次该节点选出来的那一个属性是刚刚其父节点分裂时用过的属性，则该节点已经达到了叶子节点，无须继续分裂了）。一直到不能够再分裂为止。注意整个决策树形成过程中没有进行剪枝。

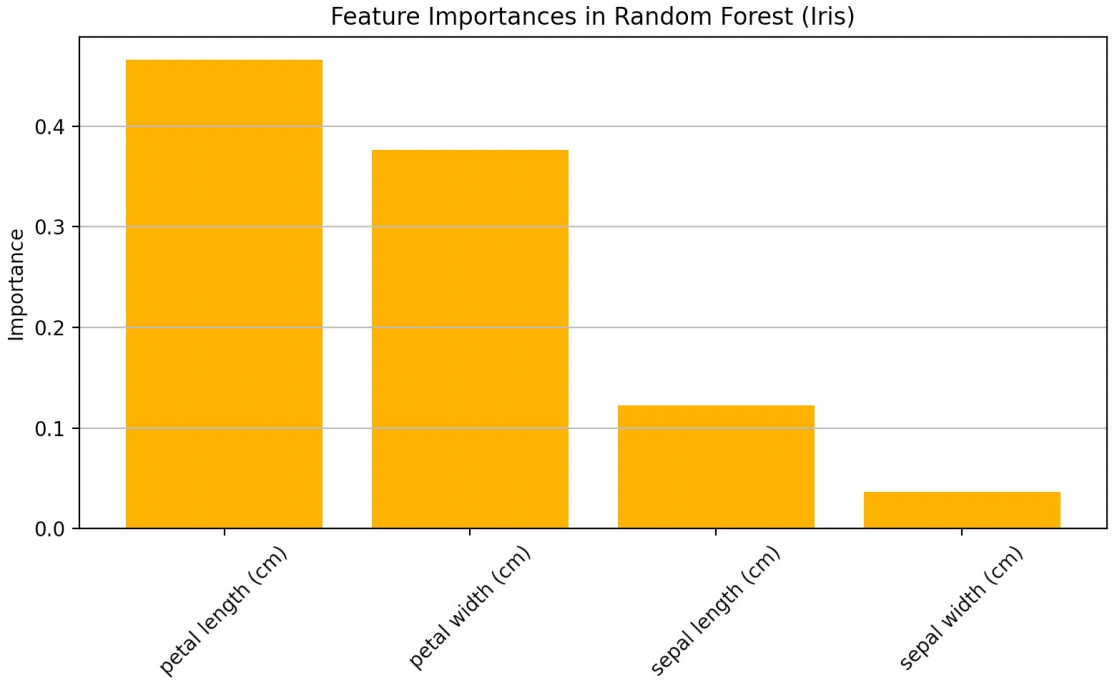
4 形成森林：按照步骤1~3建立大量的决策树，这样就构成了随机森林了。

Random Forests

一个例子

```python
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
import numpy as np

# 加载数据
iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
target_names = iris.target_names
```



可解释性：花瓣长度和宽度的重要性远大于萼片特征

Random Forests

npj | natural hazards

Article

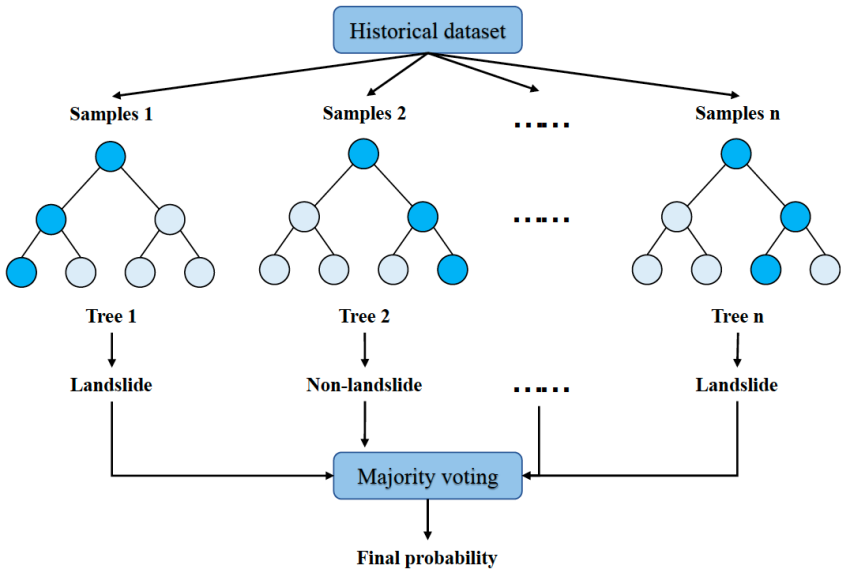
https://doi.org/10.1038/s44304-025-00146-8

Interpretable machine learning incorporating major lithology for regional landslide warning in northern and eastern Guangdong

Check for updates

Wenfeng Cui[1], Kejie Chen[1,2] ✉, Zhanhui Qing[2,3], Zhiwen Zheng[2,3], Wei Zhang[2,3], Zhihua Zhou[2,3], Peng Han[1], Junling Zhang[4], Zhaoyu Zhu[4] & Chuangeng Sun[1]

Landslides pose major risks in northern and eastern Guangdong, China, due to complex geology and heavy rainfall. Traditional models often oversimplify lithology and lack interpretability. This study develops a lithology-specific random forest model that distinguishes between sedimentary and igneous rocks and integrates rainfall, geological, and geotechnical data. Using 754 landslide cases and 1233 rainfall records, the model achieves over 90% hit rate and below 4% false alarm rate. Interpretable machine learning techniques, including feature importance rankings, SHAP values, and partial dependence plots, are used to understand how different factors contribute to landslide occurrence. A case study from the 2024 Pingyuan landslides confirms the model's real-world applicability. This framework offers improved prediction performance and interpretability and can serve as a robust tool for regional early warning and risk management in geologically diverse areas.



Bagging vs. Boosting 比较表

特征	Bagging	Boosting
🧩 核心思想	并行训练多个弱分类器，然后投票（平均）组合	顺序训练多个弱分类器，后一个模型纠正前一个模型的错误
🔄 样本选取方式	**有放回随机采样（Bootstrap）**，每轮选不同样本子集	每轮根据前一轮的错误样本 **调整权重**，关注"难分类"样本
🧠 训练方式	**并行**训练多个分类器，互不影响	**串行**逐步训练，每一轮依赖前一轮的结果
🧪 主要目标	减少模型的 **方差**（variance） 防止过拟合	减少模型的 **偏差**（bias） 提升预测能力
🎯 关注重点	所有样本都"平均对待"	对错误分类样本 **加大关注**
🌰 常见基学习器	决策树（如 CART），可用高方差模型	决策树（浅层 stump），适用于弱学习器
📦 典型代表	**Random Forest**	**AdaBoost, Gradient Boosting, XGBoost**
📈 对异常值的敏感性	不太敏感（因为投票平滑了误差）	较敏感（错误点被反复强调）
⚙️ 可并行性	✅ 高，可并行加速	❌ 串行依赖，难以并行化

Bagging： 大家一起干，投票为主，稳中求胜
Boosting： 一棒接一棒，错了就学，步步为营

For more information

scikit learn Install User Guide API Examples More Go

Prev Up Next

scikit-learn 1.0
Other versions

Please cite us if you use the software.

sklearn.ensemble.RandomForestClassifier
Examples using sklearn.ensemble.RandomForestCl

sklearn.ensemble.RandomForestClassifier

class sklearn.ensemble.RandomForestClassifier(*n_estimators=100, *, criterion='gini', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='auto', max_leaf_nodes=None, min_impurity_decrease=0.0, bootstrap=True, oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False, class_weight=None, ccp_alpha=0.0, max_samples=None*) ¶ [source]

A random forest classifier.

A random forest is a meta estimator that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting. The sub-sample size is controlled with the `max_samples` parameter if `bootstrap=True` (default), otherwise the whole dataset is used to build each tree.

Read more in the User Guide.

Parameters:

n_estimators : *int, default=100*
The number of trees in the forest.

Changed in version 0.22: The default value of `n_estimators` changed from 10 to 100 in 0.22.

criterion : *{"gini", "entropy"}, default="gini"*
The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "entropy" for the information gain. Note: this parameter is tree-specific.

max_depth : *int, default=None*
The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.

min_samples_split : *int or float, default=2*
The minimum number of samples required to split an internal node:

- If int, then consider `min_samples_split` as the minimum number.
- If float, then `min_samples_split` is a fraction and `ceil(min_samples_split * n_samples)` are the minimum number of samples for each split.

Changed in version 0.18: Added float values for fractions.

min_samples_leaf : *int or float, default=1*
The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

使用随机森林实现岩石分类

Geochemistry, Geophysics, Geosystems®

RESEARCH ARTICLE
10.1029/2021GC009946

Key Points:

- We train machine learning (ML) models to reveal the mantle source compositions of basaltic rocks
- Source lithology distributions of basalts in NE China predicted by the ML models are consistent with previous studies
- Pyroxenite and peridotite may dominate the mantle source of basaltic rocks above and off the stagnant Pacific slab, respectively

Supporting Information:

Supporting Information may be found in the online version of this article.

Correspondence to:

T. Yang,
yangt3@sustech.edu.cn

Citation:

Machine Learning Reveals Source Compositions of Intraplate Basaltic Rocks

Peng Guo[1], Ting Yang[1,2], Wen-Liang Xu[3], and Bin Chen[1]

[1]Department of Earth and Space Sciences, Southern University of Science and Technology, Shenzhen, China, [2]Shenzhen Key Laboratory of Deep Offshore Oil and Gas Exploration Technology, Southern University of Science and Technology, Shenzhen, China, [3]College of Earth Sciences, Jilin University, Changchun, China

Abstract Recycling of crustal material is thought to introduce pyroxenite to the peridotite mantle. Mapping such lithological heterogeneity within the mantle is crucial to understanding the mantle's chemical evolution but remains challenging. By sampling the mantle source, intraplate basaltic melts provide a unique chance to reveal lithological heterogeneity within the mantle. We train machine learning (ML) models with major oxide data of experimental peridotite and pyroxenite melts to help reveal the mantle source lithology of basaltic rocks. The ML models can predict source lithologies from major oxide information with an accuracy larger than 94%. As a case study, we predict source lithology of the Cenozoic intraplate basaltic rocks in Northeast China. Our ML models suggest that pyroxenite dominates the mantle source of basaltic rocks sitting above the stagnant Pacific slab while peridotite dominates the source of the basaltic rocks located west of the slab tip, consistent with previous studies using other approaches. Our ML models could potentially be used to infer mantle source lithologies of basaltic rocks from other regions around the world.